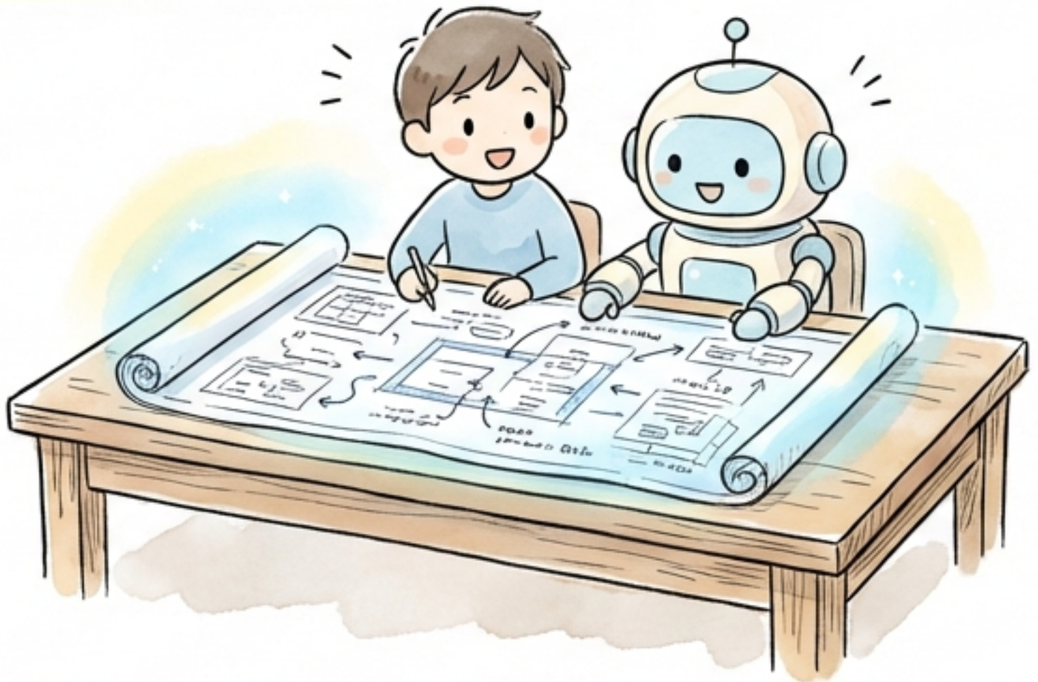
# コミュニケーションの分散を終わらせ、Cursorを最強の相棒にする方法

## DX互助会向け：コミュニケーション連携と一元管理パッケージ

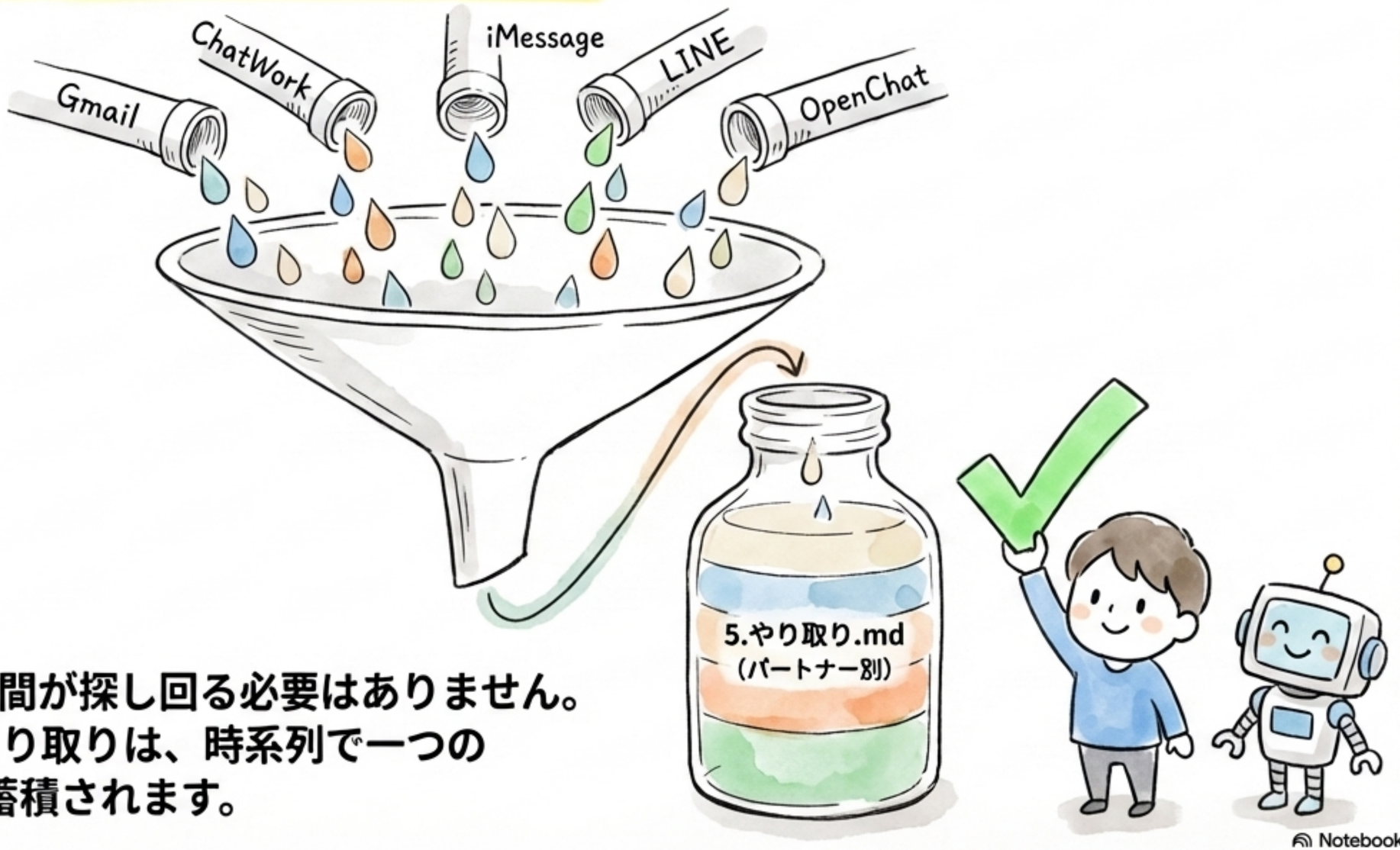


「どこにどう残し、Cursorでどう再現するか」——
実例からローカル適用パッケージまでを解き明かすアイデアノート。

NotebookLM

## 複数チャネルからの情報を、一つの「型」に流し込む

# 読む場所と取り込みの型を揃える

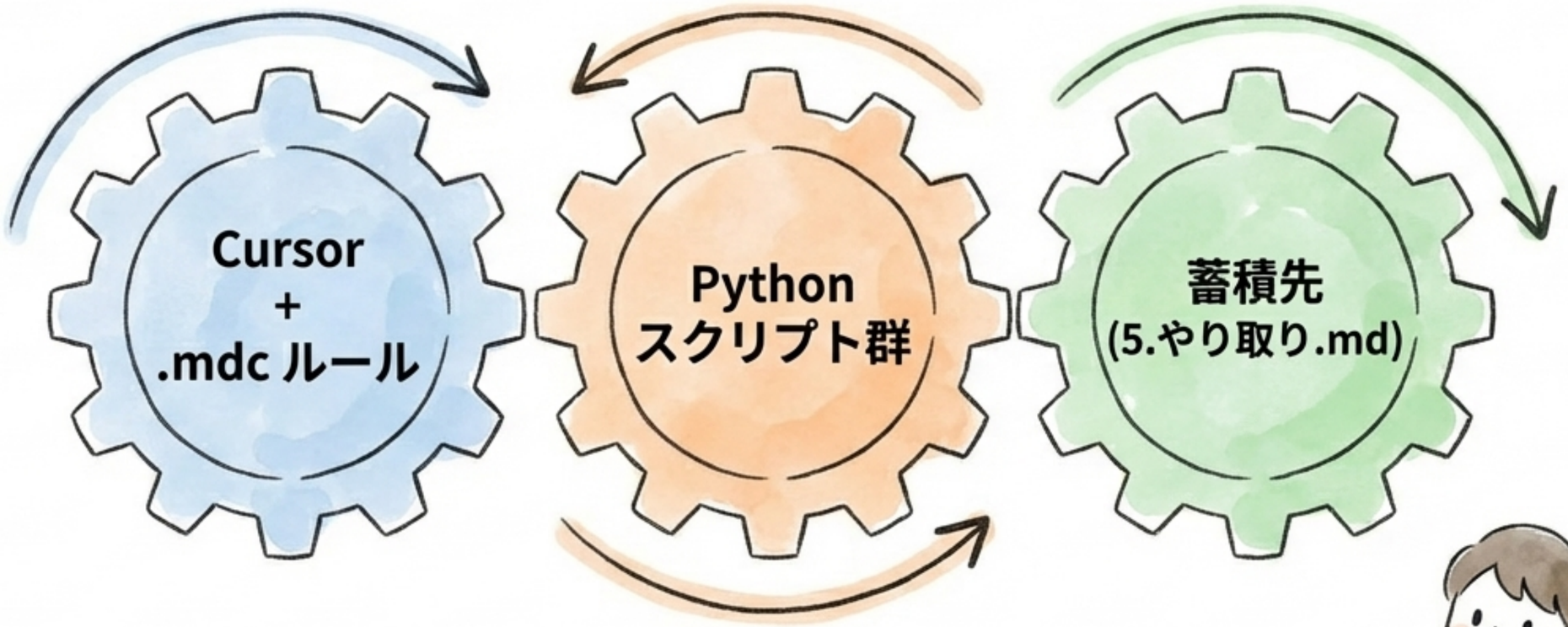



チャネルが増えても、人間が探し回る必要はありません。
すべてのパトナーとのやり取りは、時系列で一つの
Markdownファイルに蓄積されます。

NotebookLM

## ルールとスクリプトが噛み合う「自動化の歯車」

# 再現性の中核は .mdc にある




Cursorのプロジェクトルール（.mdc）に「取り込みの順序」や「送信のスクリプト」を固定することで、AIへの依頼のぶれを完全に排除します。



NotebookLM

# チャネル別・取得と制約の手引き（マトリックス）

| The Channel Integration Matrix | | | | |
|---|---|---|---|---|
| 連携チャネル | 取得方法 | 認証ハードル | 固有の制約 | 送信対応 |
| Gmail | Gmail API | 低 (OAuth) | なし | ✔あり |
| ChatWork | ChatWork API | 低 (Token) | なし | ✔あり |
| iMessage | SQLite (chat.db) | 中 (Mac限定) | フルディスクアクセス必須 | ✔あり |
| LINE 個人 | 非公式 CHRLINE | 高 (QR回避工夫) | E2EEで本文欠落リスク | ✖なし |
| LINE オプチャ | 非公式 CHRLINE | 高 | スレッド本文が欠ける場合あり | ✖なし |

すべてが完璧な公式APIではありません。非公式APIの制約（E2EEやスレッド欠落）を割り切ってでも、一元化による恩恵を取りに行く設計です。

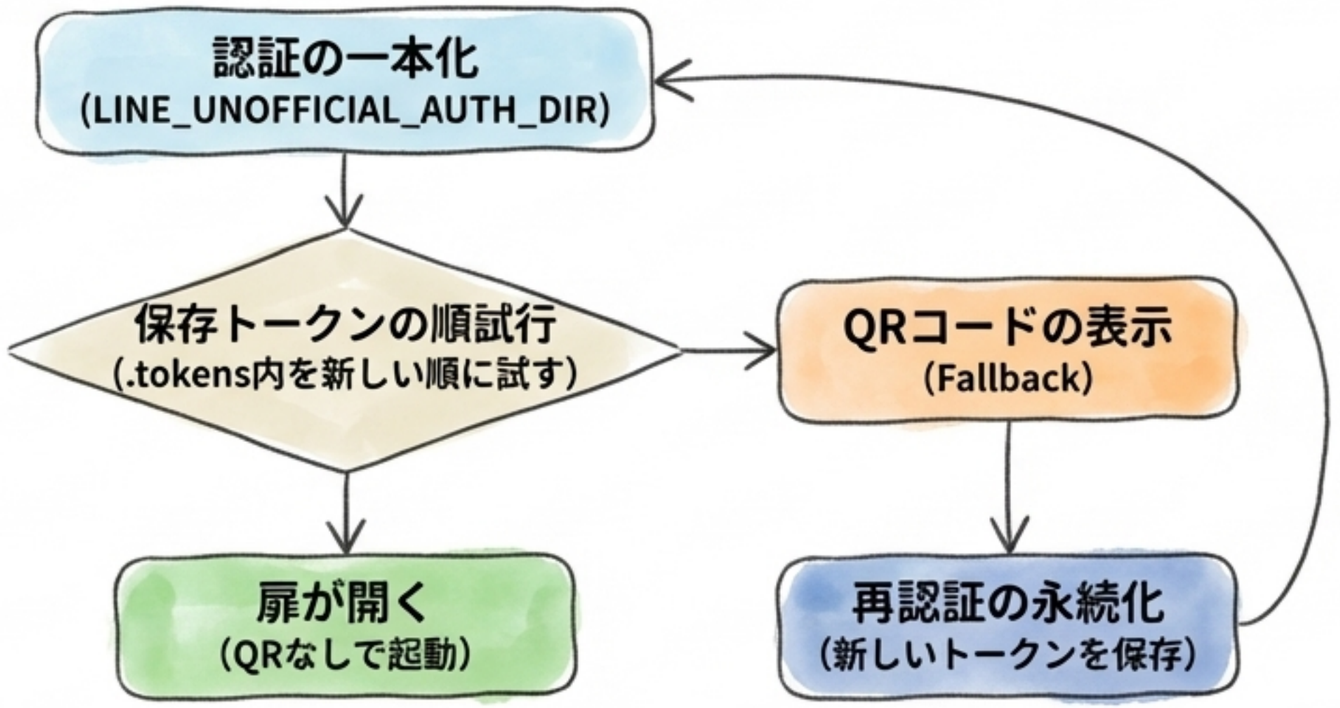
NotebookLM

Digital-Craft Sketchbook

# 泥臭い工夫：非公式LINE APIの認証を安定させる

## 「QRが連発する」を防ぐ防波堤






スクリプトごとに別ログインが発生しないよう、トーク同期とオプチャ取得で認証を共有。
古い鍵から試し、ダメな時だけ新しい鍵（QR）を作ります。

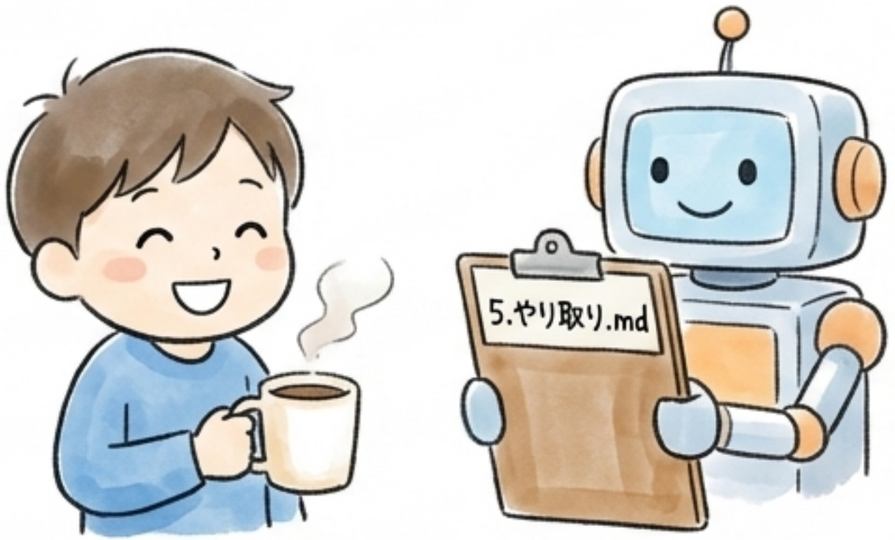
NotebookLM

Digital-Craft Sketchbook

# 「探して迷う時間」が消滅し、AIが文脈を完全把握する実運用での圧倒的な「効き」




単なる指示：「これに返信して」
(AIは文脈ゼロで無難な回答しかできない)



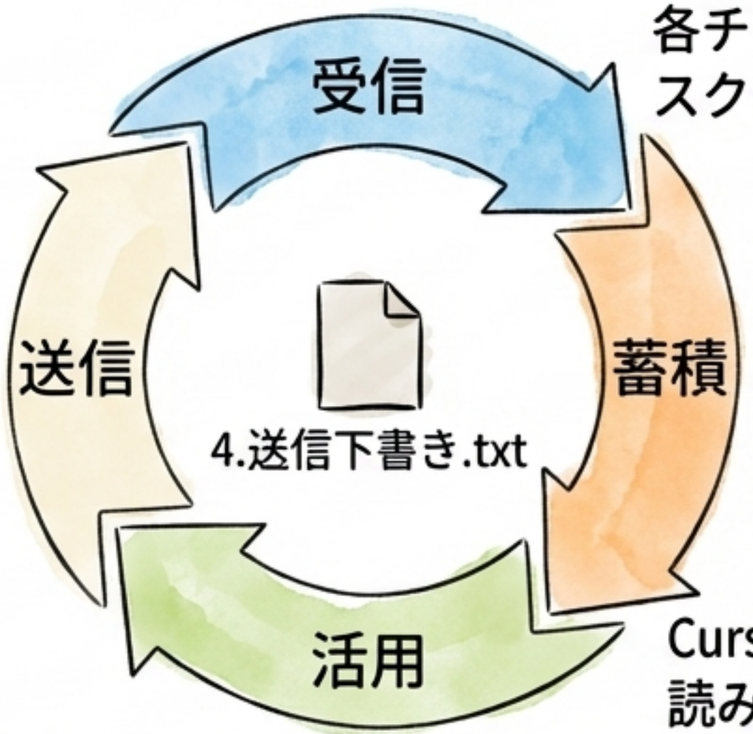

Cursorへの指示：
「パートナーからのメール等を確認して」

日々の業務が「あちこち探す」から、「全体の様子を把握し、文面のたたき台を一緒に考える」という高度なフェーズへ移行します。

NotebookLM

# 受信から送信まで、情報のエコシステムを閉じる

## 送信導線も「一つの型」に寄せる




各チャネルから
スクリプトで取り込み

5.やり取り.md
へ時系列で積み上げ

Cursorがコンテキストとして
読み込み、ドラフト作成

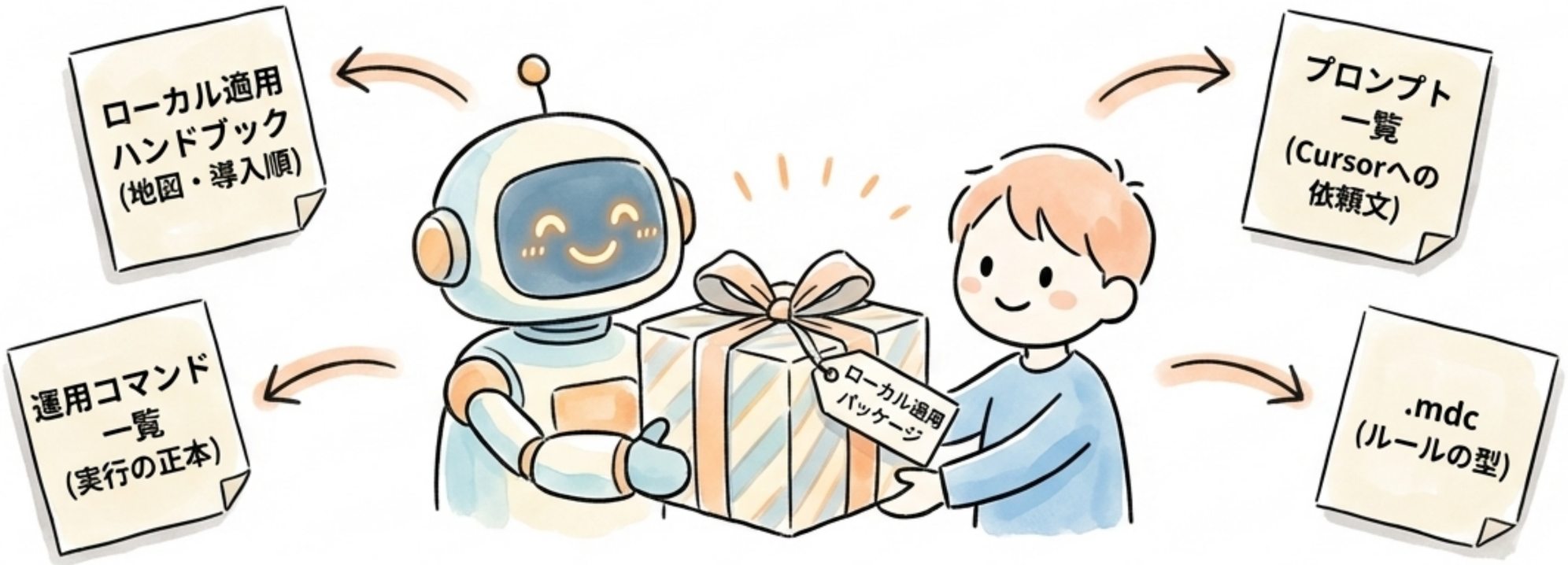
yoritoori_send.py
へ導線を完全統一

どのチャネルから来た連絡でも、返信の作成・送信は常に同じスクリプトとファイルを経由します。これにより履歴の追記漏れを防ぎます。

NotebookLM

# この強力な仕組みを、ご自身のローカル環境へ

## そのまま「パクって」試してみてください




完コピでなくても構いません。フォルダ構成は自由です。
この「導線とルールの型」を持ち帰り、あなたのワークスペースに組み込んでください。

NotebookLM

# 導入のための「3つの飛び石」



1. 置く: パッケージをワークスペースのルートに配置する。

2. 参照する: Cursorのチャットで@ハンドブックと呼び出し、「セットアップして」と依頼する。

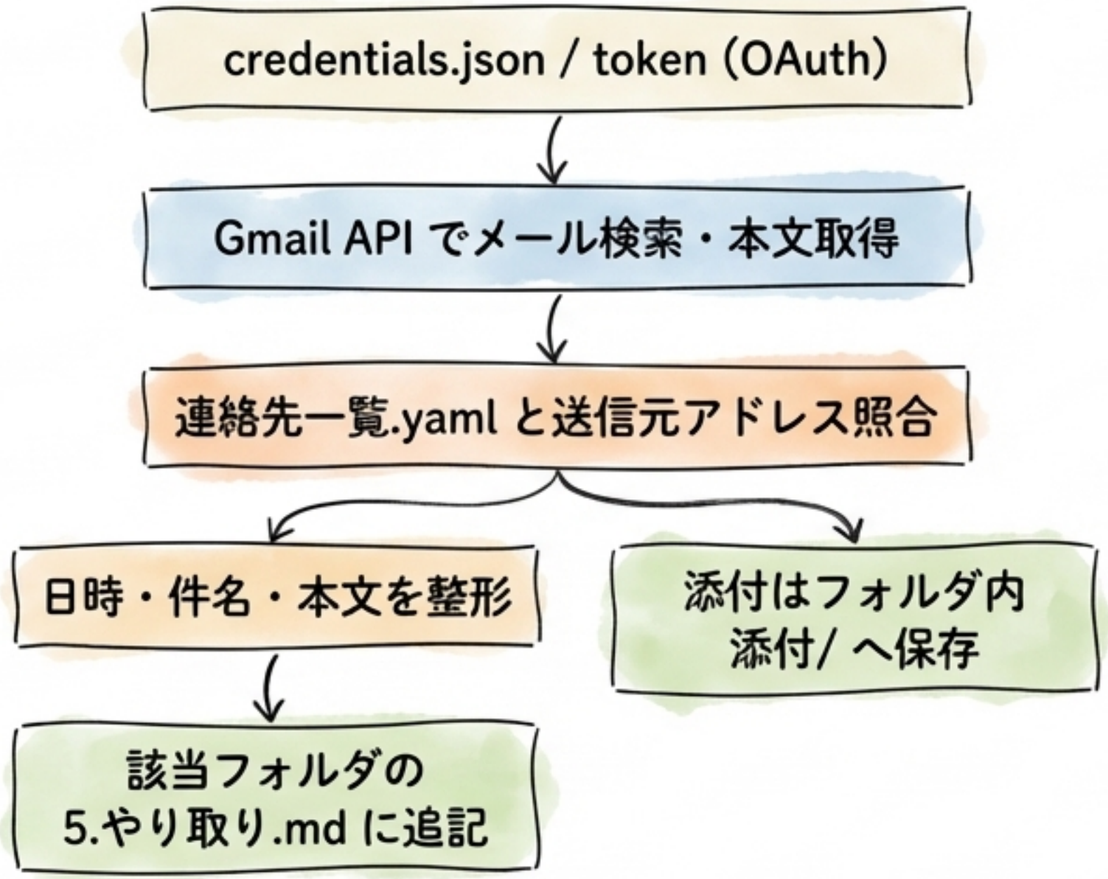
3. 合わせる: コマンド一覧のパスを、自分のPC環境に合わせて書き換える。

**Cursor自体に手伝わせることで、セットアップの手間を最小化します。**
**ルールの型さえあれば、あなたのPCですぐに動き始めます。**

NotebookLM

Digital-Craft Sketchbook

# 武器庫: Gmailの取得フロー




Gmail APIを利用した標準的な取得フローです。送信時は別スクリプト（付録F）を使用します。



NotebookLM

Digital-Craft Sketchbook

# 武器庫: ChatWorkの取得フロー

.env の CHATWORK_API_TOKEN

↓

ChatWork API でルーム・メッセージ取得

↓

連絡先一覧の chatwork_room_id と照合

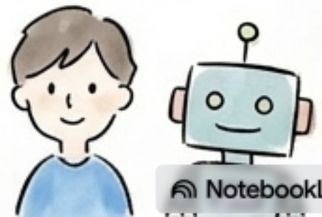
↓

整形（日時・投稿者・本文）

↓

該当フォルダの
5.やり取り.md に追記

APIトークンとルームIDを紐付けることで、特定のプロジェクトやパートナーごとの発言を正確に抽出・蓄積します。



NotebookLM

# 武器庫: iMessageの取得フロー

## Terminal/Cursorに「フルディスクアクセス」が必須

~/Library/Messages/chat.db

↓

SQLite で新着メッセージ取得

↓

連絡先一覧 phones と電話番号照合

↓

日時・本文を整形

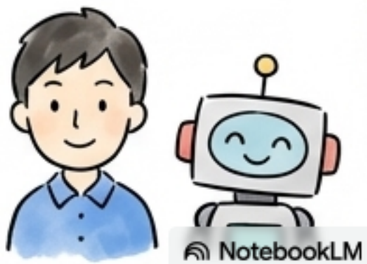
↓

5.やり取り.md に追記

↓

処理済み ROWID を JSON に保存

macOSローカルのSQLiteデータベースを直接参照するアプローチです。
既読・未読の管理は ROWIDで行います。



NotebookLM

# 武器庫: LINE（個人・グループ）取得フロー

## E2EEで本文が取れない場合あり

LINE_UNOFFICIAL_AUTH_DIR / .tokens

↓

build_logged_in_client で CHRLINE 起動

↓

同期オペレーション (メッセージ系) を取得

↓

プリセット・chatMid で対象トークに絞込

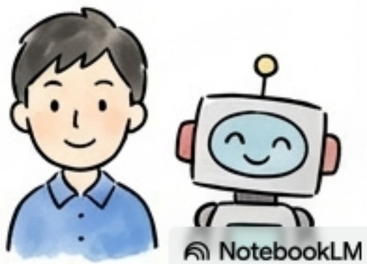
↓

dedup (既追記 ID はスキップ)

↓

該当の 5.やり取り.md に追記

プリセットで固定された対象トークからメッセージを同期します。
非公式APIの制約がある点を運用でカバーします。



NotebookLM

# 武器庫: LINEオープンチャット取得フロー

## 手動補完スクリプトも併用（Beta運用）

open_chat_routes.yaml（ルート定義）

↓

build_logged_in_client（トーク同期と共有可）

↓

Square でメイン・スレッドを取得

↓

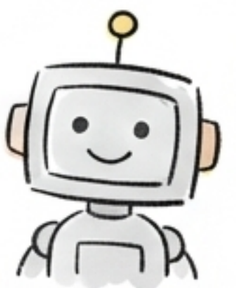
state.json で続きから / dedup で重複除外

↓

各 output_md へ追記

APIの制約で取り切れないスレッドがあるため、完全自動化ではなく
手動補完とセットのBeta版としての運用を推奨します。

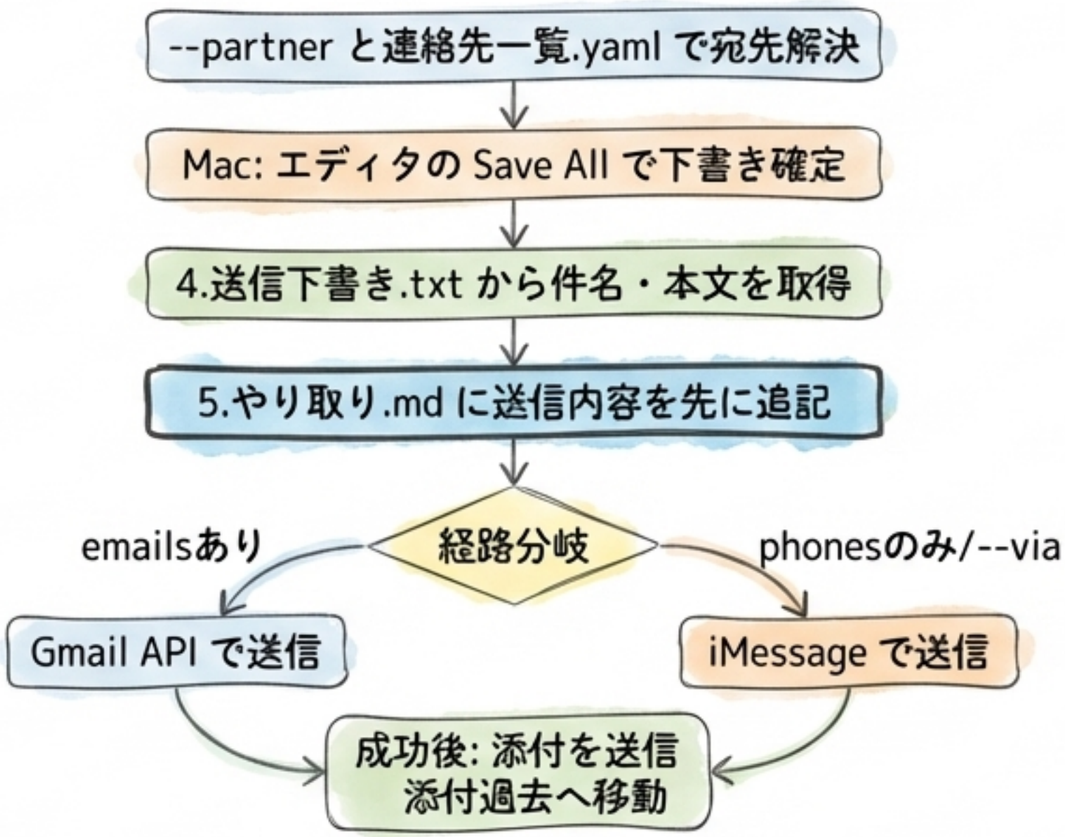




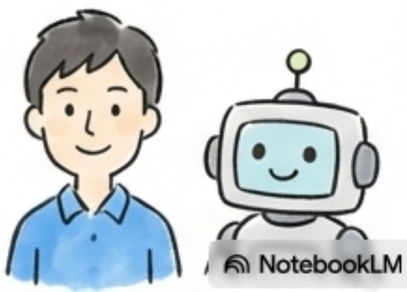
NotebookLM

# 武器庫: 統合送信フロー

## 実際に送信する前に「やり取りMD」に記録する




partner-email-send.mdc ルールにより呼び出されます。
送信前に履歴を確定させることで、ログの欠損を完全に防ぎます。



NotebookLM